Backend Module Specifications

Harmonized Shared Backend Architecture

User Story 3: POI Search & Itinerary · User Story 2: Trip Invitation & Collaboration

6 Modules Specified · Layers 2, 2B & 4 · March 2026

Module Overview

#	Module Name	Layer	Key Classes / Components
1	POI Controllers	Layer 2 — REST API	POISearchController, TripController, ItineraryController
2	Invitation Controllers	Layer 2 — REST API	InvitationController, CollaboratorController, ShareLinkController
3	Shared Middleware	Layer 2 — REST API	AuthMiddleware (JWT), RateLimiter (Redis), SessionMiddleware
4	Services	Layer 2 — Business Logic	ACIService, POIService, TripService, CollaboratorService, InvitationService, ShareableLinkService
5	Real-Time Server	Layer 2B — Socket.IO Process	PresenceService, ItineraryUpdateBroadcaster, CollaboratorEventEmitter
6	Managed Data Services	Layer 4 — Cloud Storage	PostgreSQL 16 (Prisma ORM), Redis 7.2 (ioredis)

Sections per Module

Each module is documented with all seven required sections:
1. Features (what it does / does not do) 2. Internal Architecture + Mermaid diagram 3. Data Abstraction (MIT 6.005 §13)
4. Stable Storage + schemas 5. External API 6. Class/method/field declarations 7. Mermaid class hierarchy

MODULE 1 [Layer 2 — REST API Server]

POI Controllers

HTTP request handling for POI search, trip management, and itinerary CRUD

1. Features

What this module does

- Exposes three HTTP controllers that handle all inbound REST requests related to POI search and itinerary management.

- POISearchController: accepts GET /api/pois/search, validates query parameters (text, lat/lng, radius, types), delegates to POIService, and returns a normalized POI array. This is the only unauthenticated endpoint in the system.
- TripController: handles POST /api/trips/:id/pois (add a POI to a trip) and DELETE /api/trips/:id/pois/:poiId (remove a POI). Both require a valid JWT. Delegates permission checks and business logic entirely to TripService.
- ItineraryController: handles GET /api/trips/:id/itinerary. Fetches the ordered list of ItineraryItems for a trip. Requires collaborator-level JWT.
- Each controller validates and sanitizes incoming request data using express-validator before passing a typed DTO to the service layer. Controllers never perform business logic themselves.
- Controllers set HTTP status codes and response shapes consistently: 200 for reads, 201 for creates, 204 for deletes, 400 for validation errors, 401/403 for auth failures, 404 for not-found, 409 for conflicts.

What this module does NOT do

- Does not implement business logic, permission checks, or duplicate detection — all delegated to TripService / POIService.
- Does not talk to the database directly — no Prisma calls inside controllers.
- Does not communicate with external APIs (Google Places, Yelp) directly.
- Does not emit WebSocket events — that is TripService's responsibility via Redis Pub/Sub.
- Does not manage authentication token issuance — that belongs to AuthMiddleware.

2. Internal Architecture

Each controller extends the abstract Controller base class, which provides a standard request/response wrapper and error-forwarding helper. Controllers are thin: they call express-validator to validate the request, construct a typed DTO from req.body / req.params / req.query, invoke exactly one service method, then serialize the result. A shared ErrorHandler middleware catches any thrown ServiceError and maps it to an HTTP status.

⬡ Module 1 — Internal Architecture

```
graph TD
  subgraph "Layer 1 — Client"
    C[React Frontend]
  end
  subgraph "Module 1 — POI Controllers [Layer 2]"
    Router[Express Router\n/api/pois  /api/trips]
    PSC[POISearchController\nGET /api/pois/search]
    TC[TripController\nPOST /DELETE /api/trips/:id/pois]
    IC[ItineraryController\nGET /api/trips/:id/itinerary]
    EV[express-validator\nValidation Chains]
    EH[ErrorHandler Middleware]
  end
  subgraph "Module 3 — Services [Layer 2]"
    PS[POIService]
    TS[TripService]
  end
  C -->|HTTP REST| Router
  Router --> PSC
  Router --> TC
  Router --> IC
  PSC --> EV
  TC --> EV
  IC --> EV
  EV -->|SearchQueryDTO| PS
  EV -->|AddPOIDTO / RemovePOIDTO| TS
  EV -->|TripIdDTO| TS
  PS -->|POI[]| PSC
  TS -->|ItineraryItem| TC
  TS -->|ItineraryItem[]| IC
  PSC -->|JSON 200| C
```

```
  TC -->|JSON 201/204| C
  IC -->|JSON 200| C
  EH -.->|catches ServiceError| TC
  EH -.->|catches ServiceError| PSC
```

3. Data Abstraction

MIT 6.005 Reading 13 — Data Abstraction

The key abstract data type in this module is the `SearchQueryDTO` — an immutable value object. Its rep invariant is: text is non-empty string ≤ 255 chars; lat/lng, if present, are valid WGS-84 coordinates; radius is 0–50 000 m; types is a non-empty subset of the allowed POI type enum. Clients cannot construct a SearchQueryDTO directly — they go through the controller's validation chain, which acts as the factory and enforces the RI. The abstraction function maps raw HTTP query strings onto a typed, range-checked value that the service layer can trust unconditionally.

Similarly, AddPOIDTO is an immutable value: { tripId: UUID, poi: POI, notes?: string }. It is produced by TripController only after express-validator confirms tripId is a valid UUID and poi satisfies the POI schema. Once created, neither DTO mutates — they are passed straight through to the service and discarded.

4. Stable Storage

Controllers themselves are stateless — they hold no durable state. All persistence is owned by the service layer (Module 3) and the database layer (Module 6). The only module-level "state" is the Express Router registration, which is rebuilt on process start from code.

Storage: None (stateless module)

If the controller process crashes and restarts, no data is lost because no data is stored here. All in-flight requests are dropped; the client retries. This is safe by design.

5. External API

Method	Endpoint	Auth	Request	Response
GET	/api/pois/search	None (public)	Query: text, lat?, lng?, radius?, types?	200 { results: POI[], cached: boolean }
POST	/api/trips/:id/pois	JWT — OWNER|EDITOR	Body: { poi: POI, notes?: string }	201 { item: ItineraryItem }
GET	/api/trips/:id/itinerary	JWT — Collaborator	Params: id (UUID)	200 { items: ItineraryItem[], trip: Trip }
DELETE	/api/trips/:id/pois/:poiId	JWT — OWNER or item creator	Params: id, poiId (UUIDs)	204 No Content

Error Responses (all endpoints)

Status	Code	Meaning
400	VALIDATION_ERROR	Request body / query failed express-validator rules
401	UNAUTHORIZED	JWT absent or expired
403	FORBIDDEN	Authenticated user lacks required role
404	NOT_FOUND	Trip or ItineraryItem does not exist
409	CONFLICT	POI already exists in itinerary (add endpoint)
429	RATE_LIMITED	RateLimiter middleware rejected request
500	INTERNAL	Unhandled service exception

6. Class, Method & Field Declarations

Abstract Controller (base class — package-private)

⬡ Controller base class

```
classDiagram
  class Controller {
    <<abstract>>
    #router: Router
    #wrapAsync(fn: RequestHandler): RequestHandler
    #sendSuccess(res, data, status?): void
    #sendError(res, error: ServiceError): void
    +abstract registerRoutes(): void
  }
```

POISearchController

Visibility	Member	Type / Signature	Notes
private	poiService	POIService	injected via constructor
private	validationChain	ValidationChain[]	express-validator rules for search params
public	registerRoutes()	void	mounts GET /api/pois/search on router
private	handleSearch(req, res, next)	RequestHandler	validates → builds SearchQueryDTO → calls poiService.searchPOIs()
private	buildSearchDTO(req)	SearchQueryDTO	throws ValidationError if RI violated

TripController

Visibility	Member	Type / Signature	Notes
private	tripService	TripService	injected
private	addValidation	ValidationChain[]	validates poi object + notes
private	removeValidation	ValidationChain[]	validates tripId + poiId as UUIDs
public	registerRoutes()	void	mounts POST and DELETE on /api/trips/:id/pois

Visibility	Member	Type / Signature	Notes
private	handleAddPOI(req, res, next)	RequestHandler	builds AddPOIDTO → tripService.addPOIToItinerary()
private	handleRemovePOI(req, res, next)	RequestHandler	builds RemovePOIDTO → tripService.removePOIFromItinerary()

ItineraryController

Visibility	Member	Type / Signature	Notes
private	tripService	TripService	injected
private	getValidation	ValidationChain[]	validates tripId UUID
public	registerRoutes()	void	mounts GET /api/trips/:id/itinerary
private	handleGetItinerary(req, res, next)	RequestHandler	calls tripService.getItinerary(tripId)

7. Class Hierarchy Diagram

⬡ Module 1 — Class Hierarchy

```
classDiagram
  class Controller {
    <<abstract>>
    #router: Router
    #wrapAsync(fn) RequestHandler
    #sendSuccess(res, data, status) void
    #sendError(res, err) void
    +registerRoutes()* void
  }
  class POISearchController {
    -poiService: POIService
    -validationChain: ValidationChain[]
    +registerRoutes() void
    -handleSearch(req,res,next) void
    -buildSearchDTO(req) SearchQueryDTO
  }
  class TripController {
    -tripService: TripService
    -addValidation: ValidationChain[]
    -removeValidation: ValidationChain[]
    +registerRoutes() void
    -handleAddPOI(req,res,next) void
    -handleRemovePOI(req,res,next) void
  }
  class ItineraryController {
    -tripService: TripService
    -getValidation: ValidationChain[]
    +registerRoutes() void
    -handleGetItinerary(req,res,next) void
  }
  Controller <|-- POISearchController
  Controller <|-- TripController
  Controller <|-- ItineraryController
```

MODULE 2 [Layer 2 — REST API Server]

Invitation Controllers

HTTP request handling for invitations, collaborator management, and shareable links

1. Features

What this module does

- InvitationController: handles POST /api/trips/:id/invitations (create & email an invitation) and PATCH /api/invitations/:token/accept (accept an invitation via token). Delegates to InvitationService for all business logic and email dispatch.
- CollaboratorController: handles GET /api/trips/:id/collaborators (list), PATCH /api/trips/:id/collaborators/:userId (update role), and DELETE /api/trips/:id/collaborators/:userId (remove). All mutating endpoints require OWNER role. Delegates to CollaboratorService.
- ShareLinkController: handles POST /api/trips/:id/sharelink (generate a shareable join URL) and GET /api/trips/join/:code (look up a trip by invite code and return metadata). Delegates to ShareableLinkService.
- All controllers validate input with express-validator and forward typed DTOs to their respective services. They share the same abstract Controller base class as Module 1.
- Token-based invitation acceptance (PATCH /invitations/:token/accept) is an unauthenticated endpoint — the token itself is the credential.

What this module does NOT do

- Does not send emails itself — delegates unconditionally to InvitationService → SendGrid.
- Does not compute invitation expiry or generate token hashes — that is InvitationService's responsibility.
- Does not enforce role hierarchy rules (e.g., EDITOR cannot invite with OWNER role) — CollaboratorService enforces those invariants.
- Does not manage the real-time notification when a collaborator joins — that is emitted by the Service layer via Redis Pub/Sub → Layer 2B.

2. Internal Architecture

Controllers in this module mirror Module 1's pattern: thin validation → typed DTO → service call → serialize response. The critical distinction is that InvitationController has one public (unauthenticated) endpoint (/invitations/:token/accept) and one OWNER-protected endpoint. ShareLinkController's GET /join/:code endpoint is public, while POST /sharelink requires OWNER JWT.

⬡ Module 2 — Internal Architecture

```
graph TD
  subgraph "Layer 1 — Client"
    C[React Frontend]
  end
  subgraph "Module 2 — Invitation Controllers [Layer 2]"
    R2[Express Router\n/api/trips  /api/invitations]
    INV[InvitationController]
    COL[CollaboratorController]
    SLC[ShareLinkController]
    EV2[express-validator\nValidation Chains]
  end
  subgraph "Module 3 — Services [Layer 2]"
    IS[InvitationService]
    CS[CollaboratorService]
    SLS[ShareableLinkService]
  end
  C -->|HTTP REST| R2
```

```
R2 --> INV & COL & SLC
INV --> EV2
COL --> EV2
SLC --> EV2
EV2 -->|InviteDTO| IS
EV2 -->|CollaboratorDTO| CS
EV2 -->|ShareLinkDTO| SLS
IS -->|Invitation| INV
CS -->|Collaborator[]| COL
SLS -->|ShareableLink| SLC
```

3. Data Abstraction

MIT 6.005 Reading 13 — Data Abstraction

The central ADT is InviteDTO: an immutable value object with rep invariant: tripId is a valid UUID, inviteeEmail is a valid RFC-5321 address ≤ 255 chars, role ∈ { EDITOR, VIEWER } (controllers must reject attempts to invite with OWNER role). The abstraction function maps raw POST body fields to this validated structure. Controllers produce it; services consume it. Neither layer mutates it.

InviteTokenDTO wraps a 64-character hex token string. Its sole RI is that the string matches /^[0-9a-f]{64}$/. This allows InvitationController.handleAccept() to reject malformed tokens before any database lookup, preventing unnecessary load on InvitationService.

4. Stable Storage

Controllers are stateless. All invitation and collaborator data is persisted by the service and database layers. No controller-level state survives a process restart.

5. External API

Method	Endpoint	Auth	Request	Response
POST	/api/trips/:id/invitations	JWT — OWNER|EDITOR	Body: { inviteeEmail, role }	201 { invitation: Invitation }
PATCH	/api/invitations/:token/accept	Token (no JWT)	Params: token (hex-64)	200 { trip: Trip, role: string }
GET	/api/trips/:id/collaborators	JWT — Collaborator	Params: id (UUID)	200 { collaborators: Collaborator[] }
PATCH	/api/trips/:id/collaborators/:uid	JWT — OWNER	Body: { role: string }	200 { collaborator: Collaborator }
DELETE	/api/trips/:id/collaborators/:uid	JWT — OWNER	Params: id, uid (UUIDs)	204 No Content
POST	/api/trips/:id/sharelink	JWT — OWNER	Params: id (UUID)	201 { shareableLink: ShareableLink }
GET	/api/trips/join/:code	None (public)	Params: code (12-char alphanumeric)	200 { trip: TripPreview }

6. Class, Method & Field Declarations

InvitationController

Visibility	Member	Type / Signature	Notes
private	invitationService	InvitationService	injected
private	createValidation	ValidationChain[]	validates email format + role ∈ {EDITOR,VIEWER}
private	acceptValidation	ValidationChain[]	validates token is 64-char hex
public	registerRoutes()	void	mounts POST /invitations and PATCH /invitations/:token/accept
private	handleCreate(req,res,next)	RequestHandler	builds InviteDTO → invitationService.createInvitation()
private	handleAccept(req,res,next)	RequestHandler	builds InviteTokenDTO → invitationService.acceptInvitation()

CollaboratorController

Visibility	Member	Type / Signature	Notes
private	collaboratorService	CollaboratorService	injected
public	registerRoutes()	void	mounts GET/PATCH/DELETE on /trips/:id/collaborators
private	handleList(req,res,next)	RequestHandler	tripId → collaboratorService.listCollaborators()
private	handleUpdateRole(req,res,next)	RequestHandler	builds RoleUpdateDTO → collaboratorService.updateRole()
private	handleRemove(req,res,next)	RequestHandler	builds RemoveCollaboratorDTO → collaboratorService.removeCollaborator()

ShareLinkController

Visibility	Member	Type / Signature	Notes
private	shareableLinkService	ShareableLinkService	injected
public	registerRoutes()	void	mounts POST /sharelink and GET /join/:code
private	handleGenerate(req,res,next)	RequestHandler	tripId → shareableLinkService.generateLink()
private	handleJoin(req,res,next)	RequestHandler	code → shareableLinkService.resolveCode()

7. Class Hierarchy Diagram

⬡ Module 2 — Class Hierarchy

```
classDiagram
  class Controller {
    <<abstract>>
    #router: Router
    +registerRoutes()* void
    #wrapAsync(fn) RequestHandler
    #sendSuccess(res, data, status) void
    #sendError(res, err) void
  }
  class InvitationController {
    -invitationService: InvitationService
    -createValidation: ValidationChain[]
    -acceptValidation: ValidationChain[]
    +registerRoutes() void
    -handleCreate(req,res,next) void
    -handleAccept(req,res,next) void
  }
  class CollaboratorController {
    -collaboratorService: CollaboratorService
    +registerRoutes() void
    -handleList(req,res,next) void
    -handleUpdateRole(req,res,next) void
    -handleRemove(req,res,next) void
  }
  class ShareLinkController {
    -shareableLinkService: ShareableLinkService
    +registerRoutes() void
    -handleGenerate(req,res,next) void
    -handleJoin(req,res,next) void
  }
  Controller <|-- InvitationController
  Controller <|-- CollaboratorController
  Controller <|-- ShareLinkController
```

MODULE 3 [Layer 2 — REST API Server]

Shared Middleware

Cross-cutting concerns: JWT authentication, Redis-backed rate limiting, session management

1. Features

AuthMiddleware (JWT)

- Extracts the Bearer token from the Authorization header.
- Verifies the JWT signature using the HS256 algorithm and the APPLICATION_SECRET environment variable.
- Checks the token's jti claim against Redis (auth:token:{userId}) to support server-side revocation. If the jti is absent from Redis, the token is treated as revoked and a 401 is returned.
- On success, attaches { userId, role, tripId? } to req.user and calls next().
- Exposes an optional requireRole(role: Role) factory that produces a middleware asserting req.user.role meets the minimum threshold after AuthMiddleware has run.

RateLimiter (Redis)

- Implements a sliding-window counter using Redis INCR + EXPIRE commands per key.
- Search endpoint key: ratelimit:search:{ip} — limit 30 req/min.
- Add-POI endpoint key: ratelimit:add:{userId} — limit 20 req/min.
- WebSocket handshake key: ratelimit:ws:{ip} — limit 10 connections/min.
- Returns 429 with Retry-After header when the limit is exceeded. Limits are configurable via environment variables.

SessionMiddleware

- Uses express-session backed by connect-redis to store server-side session data (last-seen trip, UI preferences) in Redis under session:{sessionId}.
- Session cookies are HttpOnly, SameSite=Strict, Secure=true in production.
- Session TTL: 24 hours; extends on each authenticated request.

What this module does NOT do

- Does not issue JWTs — token issuance is an auth endpoint outside this architecture diagram's scope.
- Does not store business data in sessions — sessions hold only UI state.
- Does not perform IP-based blocking beyond rate limiting (no geo-blocking or deny-lists).

2. Internal Architecture

Middleware functions are composable Express RequestHandlers registered globally on the app or on specific routers. AuthMiddleware and RateLimiter share a single ioredis client instance (connection-pooled) injected at startup to avoid redundant connections.

⬡ **Module 3 — Middleware Pipeline**

```
graph LR
  REQ[Inbound Request] --> SM[SessionMiddleware\nparse cookie, load session]
  SM --> RL[RateLimiter\nchecks Redis sliding window]
  RL -->|within limit| AM[AuthMiddleware\nverify JWT + jti in Redis]
  RL -->|over limit| E429[429 Too Many Requests]
```

```
  AM -->|valid| RR[requireRole guard\ncheck req.user.role]
  AM -->|invalid| E401[401 Unauthorized]
  RR -->|pass| CTRL[Controller]
  RR -->|fail| E403[403 Forbidden]
  CTRL --> RES[Response]
```

3. Data Abstraction

MIT 6.005 Reading 13 — Data Abstraction

The key ADT produced by AuthMiddleware is AuthContext: an immutable, module-internal value object attached to req.user. Its rep invariant is: userId is a valid UUID; role ∈ { OWNER, EDITOR, VIEWER }; if tripId is present, it is a valid UUID. AuthContext is write-once — no downstream middleware or controller may mutate req.user. The abstraction function maps a raw JWT payload (string claims, exp timestamp) to this typed, validated structure, discarding all claims except sub, role, trip_id, and jti.

RateLimitContext is not exposed outside this module. Internally it tracks { key: string, count: number, windowExpiry: number }. It is ephemeral (in Redis only) and never serialized into the HTTP response.

4. Stable Storage

AuthMiddleware and RateLimiter use Redis as stable storage for token revocation state and rate-limit counters respectively. Session data is also stored in Redis.

Redis Key Schema

Key Pattern	Value	TTL	Purpose
auth:token:{userId}	String — JWT jti claim	1 hour (matches token lifetime)	Token revocation check. Deleted on logout to invalidate token before expiry.
session:{sessionId}	Hash — session payload	24 hours (sliding)	express-session store. Contains userId, lastTripId, uiPrefs.
ratelimit:search:{ip}	Integer — request count	60 seconds (sliding window)	Search rate limit counter. Key auto-expires; INCR + EXPIRE pattern.
ratelimit:add:{userId}	Integer — request count	60 seconds	Add-POI rate limit counter per authenticated user.
ratelimit:ws:{ip}	Integer — connection count	60 seconds	WebSocket handshake rate limit.

5. External API

Middleware is not directly callable via REST — it is composed into the Express app pipeline. The "API" is the Express middleware contract:

Export	Signature	Usage
authMiddleware	(req, res, next) => void	Mounted globally on all protected routers
requireRole(role)	(role: Role) => Middleware	Used per-route: router.post('/pois', requireRole('EDITOR'), handler)

Export	Signature	Usage
rateLimiter(config)	(config: RateLimitConfig) => Middleware	Instantiated per endpoint with specific key/limit config
sessionMiddleware	(req, res, next) => void	Mounted at app level before all routes

6. Class, Method & Field Declarations

Visibility	Member	Type / Signature	Notes
public (export)	authMiddleware	RequestHandler	verifies JWT signature + jti in Redis; sets req.user
public (export)	requireRole(minRole)	(Role) => RequestHandler	factory; runs after authMiddleware
private	verifyToken(token)	Promise<JWTPayload>	throws AuthError on invalid sig or expiry
private	checkRevocation(jti, userId)	Promise<void>	throws AuthError if jti absent from Redis
private	redisClient	IORedis	shared singleton; injected at startup
public (export)	rateLimiter(config)	(RateLimitConfig) => RequestHandler	factory; returns sliding-window middleware
private	increment(key, windowMs)	Promise<number>	INCR + conditional EXPIRE in Redis
public (export)	sessionMiddleware	RequestHandler	express-session configured with connect-redis store

7. Class Hierarchy Diagram

⬡ Module 3 — Class Hierarchy

```
classDiagram
  class AuthMiddleware {
    <<module>>
    -redisClient: IORedis
    -jwtSecret: string
    +authMiddleware: RequestHandler
    +requireRole(minRole: Role): RequestHandler
    -verifyToken(token: string): Promise~JWTPayload~
    -checkRevocation(jti: string, uid: string): Promise~void~
  }
  class RateLimiter {
    <<module>>
    -redisClient: IORedis
    +rateLimiter(config: RateLimitConfig): RequestHandler
    -increment(key: string, windowMs: number): Promise~number~
    -buildKey(config: RateLimitConfig, req): string
  }
  class SessionMiddleware {
    <<module>>
    -redisStore: RedisStore
    +sessionMiddleware: RequestHandler
  }
  class RateLimitConfig {
    <<interface>>
    +keyFn: (req) => string
    +limit: number
    +windowMs: number
```

```
  }
  class AuthContext {
    <<value object>>
    +userId: string
    +role: Role
    +tripId?: string
  }
  RateLimiter ..> RateLimitConfig
  AuthMiddleware ..> AuthContext
```

MODULE 4 [Layer 2 — Business Logic]

Services

Core domain logic: POI aggregation, trip/itinerary management, collaboration, invitations, shareable links

1. Features

ACIService (Aggregated Content & Integration)

- Queries Google Places API and Yelp Fusion API in parallel via Promise.all for a given SearchQueryDTO.
- Deduplicates results by normalizing names and coordinates (geohash bucket within 50m). Merged entries prefer Google Places's place_id but include Yelp's ratings and review_count.
- Caches aggregated results in Redis under poi:cache:{sha256(query)} with a 1-hour TTL. Cache-aside pattern: check cache → if miss, fetch both APIs → merge → write cache → return.
- Exposes graceful degradation: if one API returns an error, logs to Sentry and returns the other source's results alone rather than failing the whole request.

POIService

- Wraps ACIService to add application-level behavior: result sorting by rating/distance, type-filtering against the allowed POI enum, and response shape normalization to the canonical POI model.
- Provides getPlaceDetails(poiId, source) to fetch rich detail (hours, photos, website) for a single POI. This is called lazily (on user click, not during search).

TripService

- addPOIToItinerary(tripId, poi, userId): checks permissions (must be OWNER or EDITOR), checks duplicate (UNIQUE constraint on trip_id + poi_id), wraps INSERT + UPDATE trips.updatedAt in a Prisma transaction, then publishes a POI_ADDED event to Redis Pub/Sub channel pubsub:trip:{tripId}.
- removePOIFromItinerary(tripId, poiId, userId): permission check (OWNER: any item; EDITOR: own item only), DELETE + UPDATE in transaction, publishes POI_REMOVED.
- getItinerary(tripId): returns ordered ItineraryItems with joined POI data; collaborator check enforced.
- broadcastUpdate(tripId, event, payload): publishes JSON to Redis pubsub:trip:{tripId}. This is the only coupling point between Layer 2 and Layer 2B.

CollaboratorService

- listCollaborators, updateRole, removeCollaborator: all enforce that the requesting user is OWNER. updateRole cannot demote the only OWNER (RI: at least one OWNER must exist per trip at all times).
- notifyCollaboratorChange(tripId, event): publishes COLLABORATOR_JOINED or COLLABORATOR_LEFT to pubsub:trip:{tripId} via Redis Pub/Sub.

InvitationService

- createInvitation: generates a 32-byte cryptographically random token, bcrypt-hashes it for storage, sets expiry 7 days from now, inserts invitation row, sends email via SendGrid using a template ID.
- acceptInvitation(rawToken): looks up invitation by token_hash, checks status === PENDING and expires_at > now(), sets status to ACCEPTED, upserts a trip_collaborators row, returns the trip and granted role.
- Tokens are single-use. Accepting sets status = ACCEPTED immediately, preventing replay.

ShareableLinkService

- generateLink(tripId): generates or retrieves the trip's invite_code (12-char alphanumeric stored on the trips table). Returns a fully-qualified URL.
- resolveCode(code): looks up the trip by invite_code, returns a TripPreview (name, destination, collaborator count). Does not automatically add the viewer as a collaborator — that requires an explicit POST after the user confirms.

What the Services module does NOT do

- Services do not speak HTTP — no req/res objects ever enter this layer.
- Services do not render emails — they provide a payload to InvitationService, which delegates to SendGrid.
- Services do not listen on WebSocket connections — they only publish to Redis Pub/Sub.

2. Internal Architecture

All services extend the abstract Service base class, which provides a transaction() helper (wrapping Prisma.$transaction), a publishEvent() helper (wrapping ioredis.publish), and a standardized ServiceError hierarchy. ACIService is called only by POIService; no controller calls ACIService directly.

⬡ Module 4 — Internal Architecture

```
graph TD
  subgraph "Controllers [Modules 1 & 2]"
    CTRL[Controllers]
  end
  subgraph "Module 4 — Services [Layer 2]"
    PS[POIService]
    ACI[ACIService]
    TS[TripService]
    CS[CollaboratorService]
    IS[InvitationService]
    SLS[ShareableLinkService]
    SE[ServiceError hierarchy]
  end
  subgraph "External [Layer 3]"
    GPA[Google Places API]
    YFA[Yelp Fusion API]
    SG[SendGrid]
  end
  subgraph "Storage [Layer 4]"
    PG[(PostgreSQL / Prisma)]
    RD[(Redis)]
  end
  CTRL --> PS & TS & CS & IS & SLS
  PS --> ACI
  ACI -->|parallel| GPA & YFA
  ACI -->|cache R/W| RD
  TS -->|transaction| PG
  TS -->|PUBLISH pubsub:trip| RD
  CS --> PG
  CS -->|PUBLISH| RD
  IS --> PG
  IS -->|email| SG
  SLS --> PG
```

3. Data Abstraction

MIT 6.005 Reading 13 — Data Abstraction

TripService is built around a mutable ADT, Trip, whose rep invariant is: exactly one OWNER exists in collaborators at all times; itinerary_items is an ordered set with no duplicate poi_id within a trip. The

abstraction function maps PostgreSQL rows (relational joins of trips + trip_collaborators + itinerary_items) to a typed Trip object. Mutators (addPOIToItinerary, removePOIFromItinerary, updateRole) enforce the RI by running inside Prisma transactions — the RI cannot be transiently violated from outside the service.

ACIService uses a cache-aside pattern. The abstract "source of truth" for POI data is the external APIs; Redis is an optimization layer. The service's contract guarantees that a cache hit and a cache miss return the same POI[] shape — callers cannot observe which path was taken.

InvitationService's Invitation ADT enforces state-machine semantics: status transitions are PENDING → ACCEPTED or PENDING → EXPIRED or PENDING → REVOKED only. No backward transitions are permitted. The service rejects any mutation that would violate this.

4. Stable Storage

PostgreSQL Tables (via Prisma ORM)

Table	Relevant Columns	Constraints	Owned By
trips	id UUID PK, name, destination, invite_code VARCHAR(12) UNIQUE, owner_id FK users, created_at, updated_at	invite_code UNIQUE; CASCADE delete → itinerary_items	TripService, ShareableLinkService
trip_collaborators	id UUID PK, trip_id FK trips, user_id FK users, role VARCHAR(20), joined_at	UNIQUE(trip_id, user_id); role ∈ {OWNER,EDITOR,VIEWER}	CollaboratorService, InvitationService
itinerary_items	id UUID PK, trip_id FK trips, poi_id VARCHAR(100), poi_data JSONB, added_by FK users, added_at, notes TEXT, order_index INT	UNIQUE(trip_id, poi_id); GIN index on poi_data	TripService
invitations	id UUID PK, trip_id FK trips, inviter_id FK users, invitee_email, token_hash VARCHAR(64) UNIQUE, role, status, expires_at, created_at	token_hash UNIQUE; status ∈ {PENDING,ACCEPTED,EXPIRED,REVOKED}	InvitationService

Redis Keys (owned by Services)

Key Pattern	Value	TTL	Owner Service
poi:cache:{sha256(query)}	JSON string — POI[]	3600s	ACIService
pubsub:trip:{tripId}	Pub/Sub channel (no stored value)	N/A — ephemeral	TripService, CollaboratorService

5. External API (Service-layer contracts for controllers)

Services are not directly exposed over HTTP — they are called by controllers. The following defines their TypeScript method signatures, which are the binding contract.

Service	Method	Parameters	Returns	Throws
POIService	searchPOIs	query: SearchQueryDTO	Promise<POI[]>	ServiceError(EXTERNAL_API_FAILED)

Service	Method	Parameters	Returns	Throws
POIService	getPlaceDetails	poiId: string, source: Source	Promise<POI>	ServiceError(NOT_FOUND)
TripService	addPOIToItinerary	tripId, poi, userId: string	Promise<ItineraryItem>	ServiceError(FORBIDDEN | CONFLICT)
TripService	removePOIFromItinerary	tripId, poiId, userId: string	Promise<void>	ServiceError(FORBIDDEN | NOT_FOUND)
TripService	getItinerary	tripId: string	Promise<ItineraryItem[]>	ServiceError(NOT_FOUND)
TripService	broadcastUpdate	tripId, event, payload: any	void	—
CollaboratorService	listCollaborators	tripId: string	Promise<Collaborator[]>	ServiceError(NOT_FOUND)
CollaboratorService	updateRole	tripId, userId, role: string	Promise<Collaborator>	ServiceError(FORBIDDEN | LAST_OWNER)
CollaboratorService	removeCollaborator	tripId, userId: string	Promise<void>	ServiceError(FORBIDDEN)
InvitationService	createInvitation	tripId, email, role, inviterId: string	Promise<Invitation>	ServiceError(ALREADY_INVITED)
InvitationService	acceptInvitation	rawToken: string	Promise<{trip,role}>	ServiceError(EXPIRED | NOT_FOUND)
ShareableLink Service	generateLink	tripId: string	Promise<ShareableLink>	ServiceError(NOT_FOUND)
ShareableLink Service	resolveCode	code: string	Promise<TripPreview>	ServiceError(NOT_FOUND)

6. Class, Method & Field Declarations

Abstract Service (base class — package-private)

Visibility	Member	Type / Signature	Notes
protected	prisma	PrismaClient	shared singleton
protected	redis	IORedis	shared singleton
protected	transaction(fn)	<T>(fn: () => Promise<T>) => Promise<T>	wraps Prisma.$transaction
protected	publishEvent(channel, payload)	(string, object) => void	wraps redis.publish with JSON.stringify
protected	log	Logger	Winston logger scoped to service name

ACIService

Visibility	Member	Type / Signature	Notes
private	googleClient	GooglePlacesClient	injected
private	yelpClient	YelpFusionClient	injected
private	cacheKeyFor(q)	(SearchQueryDTO) => string	sha256 of normalized query JSON

Visibility	Member	Type / Signature	Notes
public	search(query)	(SearchQueryDTO) => Promise<POI[]>	cache-aside; parallel API calls
private	fetchGoogle(query)	Promise<POI[]>	maps Google response to POI[]
private	fetchYelp(query)	Promise<POI[]>	maps Yelp response to POI[]
private	deduplicate(a, b)	(POI[], POI[]) => POI[]	geohash + name normalization

TripService

Visibility	Member	Type / Signature	Notes
public	addPOIToItinerary(t,p,u)	Promise<ItineraryItem>	permission + dup check + transaction + broadcast
public	removePOIFromItinerary(t,p,u)	Promise<void>	permission check + transaction + broadcast
public	getItinerary(tripId)	Promise<ItineraryItem[]>	ordered by order_index
public	broadcastUpdate(t,ev,payload)	void	redis.publish JSON to pubsub:trip:{t}
private	checkPermission(tripId,userId,minRole)	Promise<void>	throws FORBIDDEN if role < minRole
private	checkDuplicate(tripId,poiId)	Promise<void>	throws CONFLICT if UNIQUE violated

7. Class Hierarchy Diagram

⬡ Module 4 — Class Hierarchy

```
classDiagram
  class Service {
    <<abstract>>
    #prisma: PrismaClient
    #redis: IORedis
    #transaction(fn) T
    #publishEvent(channel, payload) void
    #log: Logger
  }
  class POIService {
    -aciService: ACIService
    +searchPOIs(q: SearchQueryDTO) POI[]
    +getPlaceDetails(id, src) POI
    -sortAndFilter(pois, q) POI[]
  }
  class ACIService {
    -googleClient: GooglePlacesClient
    -yelpClient: YelpFusionClient
    +search(q) POI[]
    -fetchGoogle(q) POI[]
    -fetchYelp(q) POI[]
    -deduplicate(a,b) POI[]
    -cacheKeyFor(q) string
  }
  class TripService {
    +addPOIToItinerary(t,p,u) ItineraryItem
    +removePOIFromItinerary(t,p,u) void
    +getItinerary(t) ItineraryItem[]
    +broadcastUpdate(t,ev,data) void
    -checkPermission(t,u,role) void
```

```
    -checkDuplicate(t,p) void
  }
  class CollaboratorService {
    +listCollaborators(t) Collaborator[]
    +updateRole(t,u,role) Collaborator
    +removeCollaborator(t,u) void
    -ensureNotLastOwner(t,u) void
  }
  class InvitationService {
    -mailer: SendGridClient
    +createInvitation(t,e,r,inv) Invitation
    +acceptInvitation(token) object
    -generateToken() string
    -hashToken(raw) string
    -sendEmail(inv) void
  }
  class ShareableLinkService {
    +generateLink(tripId) ShareableLink
    +resolveCode(code) TripPreview
    -ensureInviteCode(tripId) string
  }
  Service <|-- POIService
  Service <|-- ACIService
  Service <|-- TripService
  Service <|-- CollaboratorService
  Service <|-- InvitationService
  Service <|-- ShareableLinkService
  POIService --> ACIService
```

MODULE 5 [Layer 2B — Socket.IO Process]

Real-Time Server

WebSocket event fan-out, presence tracking, Redis Pub/Sub subscription for cross-process events

1. Features

PresenceService

- Tracks which users are currently online for a given trip. When a Socket.IO client emits join_trip, PresenceService writes the user to a Redis Set under presence:{tripId} (no TTL — managed by disconnect handler).
- On disconnect or leave_trip, removes the user from the Set and broadcasts a presence_update event to the trip's Socket.IO room with the updated online-user list.
- Heartbeat watchdog: if a client does not emit a ping within 30 seconds, it is treated as disconnected and removed from the presence set.

ItineraryUpdateBroadcaster

- Subscribes to Redis Pub/Sub channel pubsub:trip:* (pattern subscription) at startup. Whenever TripService (Layer 2) publishes a POI_ADDED or POI_REMOVED message, this subscriber deserializes the payload and calls socket.io.to(tripId).emit(event, payload).
- This is the only path by which itinerary mutations are broadcast to connected WebSocket clients. The REST API (Layer 2) never calls Socket.IO directly.
- Each broadcast is idempotent on the receiving client — the payload contains the full ItineraryItem, so clients can upsert rather than patch.

CollaboratorEventEmitter

- Subscribes to pubsub:trip:* for COLLABORATOR_JOINED and COLLABORATOR_LEFT events published by CollaboratorService (Layer 2).
- Broadcasts these events to the appropriate Socket.IO room so connected clients can update their collaborator sidebar in real time.
- On join_trip, emits a welcome snapshot: current presence list + last 50 ItineraryItems, giving late joiners immediate context without a separate REST call.

What this module does NOT do

- Does not write to PostgreSQL — it is read-only with respect to durable storage.
- Does not perform business logic, permission checks, or data validation on incoming events. Clients cannot mutate itinerary state over WebSocket — all mutations go through the REST API (Layers 1→2).
- Does not issue JWTs or authenticate users — it delegates token verification to AuthMiddleware's verifyToken() utility function, which is imported as a shared utility (not an HTTP middleware).
- Does not restart the Redis subscription on individual client disconnect — the subscription is process-level, not per-connection.

2. Internal Architecture

Layer 2B runs as a separate Node.js process (separate Docker container) from the REST API. It shares only the Redis instance. At startup it: (1) connects to Redis with a dedicated subscriber client (ioredis disallows mixing pub and sub on one connection), (2) pattern-subscribes to pubsub:trip:*, (3) starts the Socket.IO server on its dedicated port. The REST API (Layer 2) never calls Layer 2B's HTTP port — communication is entirely via Redis Pub/Sub.

⬡ Module 5 — Internal Architecture

```
graph TD
  subgraph "Layer 2 — REST API Process"
    TS2[TripService\nbroadcastUpdate()]
    CS2[CollaboratorService\npublishEvent()]
  end
  subgraph "Layer 4 — Redis"
    PUB["pubsub:trip:{id}\nPub/Sub Channel"]
    PRES["presence:{tripId}\nRedis Set"]
  end
  subgraph "Module 5 — Layer 2B Process"
    SUB[Redis Subscriber Client\npattern: pubsub:trip:*]
    IUB[ItineraryUpdateBroadcaster]
    CEE[CollaboratorEventEmitter]
    PRSC[PresenceService]
    SIO[Socket.IO Server]
    AUTH2B[verifyToken()\nshared utility]
  end
  subgraph "Layer 1 — Clients"
    CL[React Clients\nSocket.IO Client]
  end
  TS2 -->|PUBLISH POI_ADDED/REMOVED| PUB
  CS2 -->|PUBLISH COLLAB_JOINED/LEFT| PUB
  PUB -->|pmessage| SUB
  SUB --> IUB & CEE
  IUB -->|emit POI_ADDED| SIO
  CEE -->|emit COLLAB_JOINED| SIO
  CL -->|WSS join_trip + JWT| SIO
  SIO -->|verifyToken| AUTH2B
  SIO -->|join_trip| PRSC
  PRSC -->|SADD/SREM| PRES
  PRSC -->|emit presence_update| SIO
  SIO -->|broadcast| CL
```

3. Data Abstraction

MIT 6.005 Reading 13 — Data Abstraction

The key ADT in this module is TripRoom: a mutable set of Socket.IO socket IDs grouped by tripId, paired with an authoritative presence set in Redis. The rep invariant is: every socketId in the in-memory Socket.IO room has a corresponding entry in the Redis presence:{tripId} Set. The abstraction function maps (Socket.IO room membership ∩ Redis Set) to an ordered list of online User objects. Mutations (join/leave) maintain the RI atomically by always updating both stores before broadcasting.

BroadcastMessage is an immutable value object: { event: string, tripId: string, payload: unknown, timestamp: number }. It is produced by the Redis subscription handler and consumed by ItineraryUpdateBroadcaster.broadcast() and CollaboratorEventEmitter.emit(). Neither mutates it. The timestamp allows clients to discard out-of-order duplicates during reconnection.

4. Stable Storage

Layer 2B writes only to Redis (presence state). Itinerary data is authoritative in PostgreSQL — Layer 2B never writes there. If the Layer 2B process crashes, presence state is lost from Redis, but clients reconnect and re-hydrate from the REST API (GET /api/trips/:id/itinerary). This is acceptable: presence is ephemeral by nature.

Redis Keys (owned by Layer 2B)

Key Pattern	Value	TTL	Notes
presence:{tripId}	Redis Set of userId strings	None (managed by connect/disconnect)	Removed when user disconnects. SCARD gives online count. SMEMBERS gives user list.
presence:{tripId}:{userId}:heartbeat	String "1"	35 seconds (refreshed on ping)	Heartbeat watchdog. Absence triggers forcible removal from presence Set.

5. External API

Layer 2B exposes a Socket.IO event API (not REST). Clients connect to the Layer 2B host:port over WSS.

Direction	Event	Payload	Auth	Description
Client → Server	join_trip	{ tripId: string, token: string }	JWT in payload	Join a trip room. Server verifies token, adds to presence, sends snapshot.
Client → Server	leave_trip	{ tripId: string }	JWT in socket auth	Leave a trip room. Server removes from presence.
Client → Server	ping	{ tripId: string }	JWT in socket auth	Heartbeat. Server resets heartbeat TTL in Redis.
Server → Client	POI_ADDED	{ item: ItineraryItem, addedBy: string }	—	A collaborator added a POI. Client should upsert into local itinerary state.
Server → Client	POI_REMOVED	{ poiId: string, removedBy: string }	—	A collaborator removed a POI. Client removes from local state.
Server → Client	presence_update	{ online: User[] }	—	Current online-user list for the trip room changed.
Server → Client	COLLABORATOR_JOINED	{ user: User, role: string }	—	A new collaborator accepted an invitation.
Server → Client	COLLABORATOR_LEFT	{ userId: string }	—	A collaborator was removed from the trip.
Server → Client	snapshot	{ items: ItineraryItem[], online: User[] }	—	Sent once on join_trip. Gives late joiners current state.
Server → Client	error	{ code: string, message: string }	—	Auth failure, invalid tripId, or rate-limit exceeded.

6. Class, Method & Field Declarations

PresenceService

Visibility	Member	Type / Signature	Notes
private	redis	IORedis	standard (non-subscriber) client for writes

Visibility	Member	Type / Signature	Notes
private	io	Server	Socket.IO server instance
public	handleJoin(socket, tripId)	(Socket, string) => Promise<void>	SADD presence:{tripId} userId; emit snapshot
public	handleLeave(socket, tripId)	(Socket, string) => Promise<void>	SREM presence:{tripId} userId; broadcast presence_update
public	handleDisconnect(socket)	(Socket) => Promise<void>	removes user from all trip presence sets
public	handlePing(socket, tripId)	(Socket, string) => void	resets heartbeat:{tripId}:{userId} TTL in Redis
private	broadcastPresence(tripId)	(string) => Promise<void>	fetches SMEMBERS, emits presence_update to room
private	getOnlineUsers(tripId)	(string) => Promise<User[]>	SMEMBERS presence:{tripId} → User[]

ItineraryUpdateBroadcaster

Visibility	Member	Type / Signature	Notes
private	io	Server	Socket.IO server instance
public	handleMessage(channel, message)	(string, string) => void	called by Redis subscriber on pmessage; routes by event field
private	broadcast(tripId, event, payload)	(string, string, object) => void	io.to(tripId).emit(event, payload)
private	parseMessage(msg)	(string) => BroadcastMessage	JSON.parse + schema validation; drops malformed messages

CollaboratorEventEmitter

Visibility	Member	Type / Signature	Notes
private	io	Server	Socket.IO server instance
private	prisma	PrismaClient	read-only; used to fetch User record for snapshot
public	handleMessage(channel, message)	(string, string) => void	routes COLLABORATOR_JOINED / COLLABORATOR_LEFT events
public	sendSnapshot(socket, tripId)	(Socket, string) => Promise<void>	fetches last 50 items + online users; emits snapshot event
private	emitToRoom(tripId, event, payload)	(string, string, object) => void	io.to(tripId).emit(event, payload)

7. Class Hierarchy Diagram

⬡ Module 5 — Class Hierarchy

```
classDiagram
  class RealtimeHandler {
    <<abstract>>
    #io: Server
    #redis: IORedis
    +handleMessage(channel: string, msg: string) void
```

```
    #emitToRoom(tripId: string, event: string, payload: object) void
  }
  class PresenceService {
    +handleJoin(socket, tripId) Promise~void~
    +handleLeave(socket, tripId) Promise~void~
    +handleDisconnect(socket) Promise~void~
    +handlePing(socket, tripId) void
    -broadcastPresence(tripId) Promise~void~
    -getOnlineUsers(tripId) Promise~User[]~
  }
  class ItineraryUpdateBroadcaster {
    +handleMessage(channel, msg) void
    -broadcast(tripId, event, payload) void
    -parseMessage(msg) BroadcastMessage
  }
  class CollaboratorEventEmitter {
    -prisma: PrismaClient
    +handleMessage(channel, msg) void
    +sendSnapshot(socket, tripId) Promise~void~
    -emitToRoom(tripId, event, payload) void
  }
  class BroadcastMessage {
    <<value object>>
    +event: string
    +tripId: string
    +payload: unknown
    +timestamp: number
  }
  RealtimeHandler <|-- PresenceService
  RealtimeHandler <|-- ItineraryUpdateBroadcaster
  RealtimeHandler <|-- CollaboratorEventEmitter
  ItineraryUpdateBroadcaster ..> BroadcastMessage
  CollaboratorEventEmitter ..> BroadcastMessage
```

MODULE 6 [Layer 4 — PostgreSQL + Redis]

Managed Data Services

Durable relational storage and high-speed ephemeral cache / messaging backbone

1. Features

PostgreSQL (via Prisma ORM)

- Durable relational storage for all domain entities: Users, Trips, Collaborators, Invitations, Itinerary & ItineraryItems, POIs (cached records), SearchQueries, Destinations.
- Prisma ORM provides: type-safe queries generated from schema.prisma, connection pooling (PgBouncer-compatible), migration management (prisma migrate), and a transaction API used by TripService and InvitationService.
- JSONB column on itinerary_items.poi_data stores flexible POI schema (name, address, rating, photos, source metadata) with a GIN index for efficient querying by POI attributes.
- CASCADE delete rules: deleting a trip cascades to itinerary_items and invitations. Deleting a user sets itinerary_items.added_by to NULL (SET NULL) to preserve history while anonymizing the actor.
- Row-level security is NOT used — access control is enforced in the service layer. The database user has full CRUD on all tables but is isolated within the application's connection pool.

Redis (via ioredis)

- Five distinct logical uses — each using a namespaced key pattern to prevent collisions:
  - 1. Auth token registry: auth:token:{userId} — jti claim, 1h TTL. Token revocation support.
  - 2. Session store: session:{sessionId} — connect-redis managed, 24h sliding TTL.
  - 3. Rate limit counters: ratelimit:* — per-IP or per-user integer counters, 60s TTL.
  - 4. Presence state: presence:{tripId} — Redis Set of userId strings. No TTL; managed by connect/disconnect handlers.
  - 5. Pub/Sub channels: pubsub:trip:{tripId} — ephemeral, no stored value. Bridges Layer 2 → Layer 2B.
  - 6. POI search cache: poi:cache:{sha256} — serialized POI[], 1h TTL. Cache-aside.

What this module does NOT do

- PostgreSQL does not enforce business rules beyond FK constraints and UNIQUE indexes. Role hierarchy, permission logic, and state machine enforcement are in the service layer.
- Redis is not used as a primary database for any domain entity. If Redis is flushed, the system degrades gracefully: auth requires re-login, rate limits reset, presence is rebuilt on reconnect, POI searches hit the external APIs.
- Redis does not replicate PostgreSQL data — it is not a read replica. Queries that require joins or aggregations go to PostgreSQL.

2. Internal Architecture

PostgreSQL is accessed exclusively via Prisma ORM — no raw SQL queries are written in application code. A single PrismaClient singleton is shared across all services via dependency injection. Redis is accessed via two ioredis client instances: one standard client (for get/set/publish) and one dedicated subscriber client (for Layer 2B's pattern subscription). These two clients must never be mixed on a single connection due to ioredis subscriber mode restrictions.

⬡ Module 6 — Storage Architecture

```
graph TD
```

```
  subgraph "Application Processes"
    L2[Layer 2 — REST API\nPrismaClient singleton\nIORedis standard client]
    L2B[Layer 2B — Socket.IO\nIORedis subscriber client\nIORedis standard client]
  end
  subgraph "Module 6 — Layer 4"
    subgraph "PostgreSQL 16.1 (managed cloud)"
      PG_U[(users)]
      PG_T[(trips)]
      PG_C[(trip_collaborators)]
      PG_I[(itinerary_items)]
      PG_INV[(invitations)]
    end
    subgraph "Redis 7.2 (managed cloud)"
      R_AUTH[auth:token:*]
      R_SESS[session:*]
      R_RL[ratelimit:*]
      R_PRES[presence:*]
      R_PSUB[pubsub:trip:*]
      R_CACHE[poi:cache:*]
    end
  end
  L2 -->|Prisma ORM SQL| PG_U & PG_T & PG_C & PG_I & PG_INV
  L2 -->|ioredis GET/SET/PUBLISH| R_AUTH & R_SESS & R_RL & R_CACHE
  L2 -->|PUBLISH| R_PSUB
  L2B -->|SUBSCRIBE pattern| R_PSUB
  L2B -->|SADD/SREM/SMEMBERS| R_PRES
  L2B -->|Prisma ORM read-only| PG_I
```

3. Data Abstraction

MIT 6.005 Reading 13 — Data Abstraction

At the data layer, the key abstraction is the Database interface (DataStore ADT). Its rep invariant is a set of relational invariants enforced by PostgreSQL: referential integrity via FK constraints, uniqueness via UNIQUE indexes, and non-nullability via NOT NULL columns. The abstraction function maps Prisma-generated TypeScript objects (strongly typed by schema.prisma) to domain model instances consumed by the service layer.

Crucially, the Database module is opaque to callers: services call Prisma methods and receive typed objects — they never see raw SQL, connection strings, or transaction handles (these are encapsulated in Service.transaction()). The representation (PostgreSQL tables and indexes) can be swapped out (e.g., to CockroachDB) without changing service layer code, as long as Prisma continues to emit the same TypeScript types.

Redis's abstraction is weaker intentionally: it is a key-value store, not an object store. Each subsystem (AuthMiddleware, RateLimiter, ACIService, PresenceService) defines its own key namespace and value schema. The only shared invariant is key naming convention: all keys follow {namespace}:{discriminator}, ensuring no cross-subsystem collisions.

4. Stable Storage — Full Schema Definitions

PostgreSQL Schema (Prisma SDL)

⬡ **schema.prisma — complete model definitions**

```
model User {
  id            String    @id @default(uuid())
  email         String    @unique
  displayName   String    @db.VarChar(100)
  passwordHash  String    @db.VarChar(60)
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt
  ownedTrips    Trip[]    @relation("TripOwner")
  collaborations TripCollaborator[]
```

```
  addedItems    ItineraryItem[]
  sentInvitations Invitation[] @relation("Inviter")
}

model Trip {
  id            String    @id @default(uuid())
  name          String    @db.VarChar(200)
  destination   String    @db.VarChar(200)
  startDate     DateTime?
  endDate       DateTime?
  inviteCode    String    @unique @db.VarChar(12)
  ownerId       String
  owner         User      @relation("TripOwner", fields:[ownerId], references:[id])
  collaborators TripCollaborator[]
  items         ItineraryItem[]
  invitations   Invitation[]
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt
}

model TripCollaborator {
  id        String   @id @default(uuid())
  tripId    String
  userId    String
  role      String   @db.VarChar(20)
  joinedAt  DateTime @default(now())
  trip      Trip     @relation(fields:[tripId], references:[id], onDelete:Cascade)
  user      User     @relation(fields:[userId], references:[id], onDelete:SetNull)
  @@unique([tripId, userId])
}

model ItineraryItem {
  id          String   @id @default(uuid())
  tripId      String
  poiId       String   @db.VarChar(100)
  poiData     Json
  addedById   String?
  addedAt     DateTime @default(now())
  notes       String?  @db.Text
  orderIndex  Int      @default(0)
  trip        Trip     @relation(fields:[tripId], references:[id], onDelete:Cascade)
  addedBy     User?    @relation(fields:[addedById], references:[id], onDelete:SetNull)
  @@unique([tripId, poiId])
  @@index([tripId])
}

model Invitation {
  id           String   @id @default(uuid())
  tripId       String
  inviterId    String
  inviteeEmail String   @db.VarChar(255)
  tokenHash    String   @unique @db.VarChar(64)
  role         String   @db.VarChar(20)
  status       String   @db.VarChar(20)  // PENDING|ACCEPTED|EXPIRED|REVOKED
  expiresAt    DateTime
  createdAt    DateTime @default(now())
  trip         Trip     @relation(fields:[tripId], references:[id], onDelete:Cascade)
  inviter      User     @relation("Inviter", fields:[inviterId], references:[id])
  @@index([tokenHash])
}
```

PostgreSQL Indexes

Table	Index	Type	Purpose
users	users.email	UNIQUE B-tree	Login lookup
trips	trips.owner_id	B-tree	Trips-by-owner queries
trips	trips.invite_code	UNIQUE B-tree	Join-by-code lookup (ShareableLinkService)
trip_collaborators	(trip_id, user_id)	UNIQUE B-tree	Prevents duplicate collaborator; fast member lookup
itinerary_items	itinerary_items.trip_id	B-tree	Get all items for a trip
itinerary_items	(trip_id, poi_id)	UNIQUE B-tree	Duplicate-POI prevention
itinerary_items	poi_data	GIN	JSONB attribute queries (filter by POI type, rating)
invitations	invitations.token_hash	UNIQUE B-tree	O(1) token lookup during accept flow

Redis Key Namespace Reference

Key Pattern	Value Type	TTL	Owner	Eviction Policy
auth:token:{userId}	String (jti UUID)	3600s	AuthMiddleware	allkeys-lru fallback; explicit DEL on logout
session:{sessionId}	Hash (JSON session)	86400s sliding	SessionMiddleware	DEL on explicit logout; TTL extension on activity
ratelimit:search:{ip}	String (integer)	60s	RateLimiter	Auto-expire; no explicit delete needed
ratelimit:add:{userId}	String (integer)	60s	RateLimiter	Auto-expire
ratelimit:ws:{ip}	String (integer)	60s	RateLimiter	Auto-expire
presence:{tripId}	Set<userId>	None	PresenceService	SREM on disconnect; no TTL
presence:{tripId}:{userId}:heartbeat	String "1"	35s	PresenceService	Auto-expire triggers watchdog cleanup
pubsub:trip:{tripId}	Pub/Sub channel	N/A	TripService / CollaboratorService	Ephemeral — no stored data
poi:cache:{sha256(q)}	String (JSON POI[])	3600s	ACIService	allkeys-lru; safe to evict (cold miss hits APIs)

5. External API

Layer 4 is not directly accessible over a network interface — it is accessed exclusively by the application processes via Prisma ORM (TCP to PostgreSQL) and ioredis (TCP to Redis). There is no REST or GraphQL API at this layer. The "external API" is the Prisma Client TypeScript interface.

Interface	Operation	Example Call	Notes
Prisma	findUnique	prisma.trip.findUnique({ where: { id } })	Type-safe; throws if not found with rejectOnNotFound
Prisma	create	prisma.itineraryItem.create({ data: {...} })	Returns created row as typed object
Prisma	update	prisma.trip.update({ where: { id }, data: { updatedAt: now } })	Optimistic; throws P2025 if record missing
Prisma	delete	prisma.itineraryItem.delete({ where: { id } })	Cascades per schema config
Prisma	$transaction	prisma.$transaction([op1, op2])	Atomic; rolls back all ops on any error
ioredis	get / set / del	redis.set(key, val, "EX", 3600)	Standard Redis commands
ioredis	publish	redis.publish(channel, JSON.stringify(payload))	Pub/Sub; returns subscriber count
ioredis	subscribe / psubscribe	sub.psubscribe("pubsub:trip:*")	Subscriber client only; blocks for messages
ioredis	sadd / srem / smembers	redis.sadd("presence:"+tripId, userId)	Used by PresenceService

6. Class, Method & Field Declarations

Layer 4 is not composed of application classes — it is a managed infrastructure layer. The application-side wrappers are:

Visibility	Member	Type / Signature	Notes
public (singleton)	prisma	PrismaClient	Created once at app startup; shared via DI container
public (singleton)	redisStandard	IORedis	Standard client for GET/SET/PUBLISH; shared by Layer 2 services
public (singleton)	redisSubscriber	IORedis	Dedicated subscriber client; used exclusively by Layer 2B
public	connectDatabase()	() => Promise<void>	Calls prisma.$connect(); throws on failure; called at startup
public	disconnectDatabase()	() => Promise<void>	Calls prisma.$disconnect(); called on SIGTERM
public	connectRedis()	() => Promise<IORedis>	Verifies Redis connectivity via PING; returns client
private	handlePrismaError(e)	(PrismaClientKnownRequestError) => ServiceError	Maps Prisma error codes to ServiceError subtypes

7. Class Hierarchy Diagram

⬡ Module 6 — Class Hierarchy

```
classDiagram
  class DataStore {
```

```
    <<interface>>
    +create(data: T): Promise~T~
    +findById(id: string): Promise~T~
    +update(id: string, data: Partial~T~): Promise~T~
    +delete(id: string): Promise~void~
    +transaction(fn: ()=>Promise~T~): Promise~T~
  }
  class PrismaDataStore {
    <<singleton>>
    -client: PrismaClient
    +connectDatabase(): Promise~void~
    +disconnectDatabase(): Promise~void~
    +create(data) Promise~T~
    +findById(id) Promise~T~
    +update(id, data) Promise~T~
    +delete(id) Promise~void~
    +transaction(fn) Promise~T~
    -handlePrismaError(e) ServiceError
  }
  class RedisClient {
    <<singleton>>
    -standardClient: IORedis
    -subscriberClient: IORedis
    +connectRedis(): Promise~IORedis~
    +get(key) Promise~string|null~
    +set(key, val, ttl?) Promise~void~
    +del(key) Promise~void~
    +publish(channel, msg) Promise~number~
    +psubscribe(pattern) void
    +sadd(key, ...members) Promise~number~
    +srem(key, ...members) Promise~number~
    +smembers(key) Promise~string[]~
  }
  DataStore <|.. PrismaDataStore
```